

@tetherto/wdk-protocol-fiat-transak

This package is in beta. Please test in a dev setup first.

Builds Transak widget URLs and quotes for on-ramp (buy) and off-ramp (sell) flows. Works in both frontend and backend code.

Usage

Use the guides below for setup, trading, and transaction follow-up.

Guide	What it covers
Get Started	Install the package and initialize <code>TransakProtocol</code> .
Buy and Sell	On-ramp, off-ramp, quotes, supported assets, widget options, recipients.
Manage Transactions	Check status and load transaction details from Transak.
Configuration	API keys, caching, and Transak configuration options.
API Reference	Constructor, methods, and types for <code>TransakProtocol</code> .
Node.js Quickstart	Get started with WDK in a Node.js environment.

About WDK

Part of WDK (Wallet Development Kit) — tools for building safe, non-custodial wallets. Read more at <https://docs.wallet.tether.io>.

Get Started

Installation

```
npm install @tetherto/wdk-protocol-fiat-transak
```

Initialize TransakProtocol

Create an instance with an optional wallet account and a config object:

```
import TransakProtocol from '@tetherto/wdk-protocol-fiat-transak'

const transak = new TransakProtocol(account, {
  apiKey: 'YOUR_TRANSAK_PARTNER_KEY',
  widgetUrl, // required for buy()/sell()
  getOrder, // required for getTransactionDetail()
  environment: 'STAGING' // 'PRODUCTION' (default) | 'STAGING'
})
```

- **account** — an optional WDK wallet account (`IWalletAccount` / `IWalletAccountReadOnly`), or `undefined`. Used only by `buy/sell` to auto-fill the wallet address (see Buy and Sell).
- **config** — see Configuration for all options.

`widgetUrl` and `getOrder` are callbacks that run on **your backend**, where your API secret is safe. `widgetUrl` turns the assembled `widgetParams` into a widget URL:

```
const widgetUrl = async (widgetParams) => {
  const res = await fetch('https://your-backend.example.com/transak/widget-url', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ widgetParams })
  })
  if (!res.ok) throw new Error(`Failed to create Transak widget URL: ${res.status}`)
  const { widgetUrl } = await res.json()
  return widgetUrl
}
```

`getOrder` fetches a Transak order by its id:

```
const getOrder = async (txId) => {
  const res = await fetch(`https://your-backend.example.com/transak/order/${txId}`)
  if (!res.ok) throw new Error(`Failed to fetch Transak order: ${res.status}`)
  const { order } = await res.json()
  return order
}
```

See Creating the widget URL (backend) and Fetching an order (backend) for what those backend endpoints do.

Buy and Sell

`buy/sell` return a Transak widget URL; `quoteBuy/quoteSell` preview the economics without opening the widget.

Amounts are in **base units** (`number` | `bigint`) — fiat in cents (`10_000n` = \$100), crypto in on-chain base units (`1_000_000_000_000_000_000n` = 1 ETH). Quotes return the same way (`bigint` base units; `rate` a decimal string).

```
// Buy (on-ramp)
const { buyUrl } = await transak.buy({
  cryptoAsset: 'ETH',
  fiatCurrency: 'USD',
  fiatAmount: 10_000n,    // $100 in cents — or pass cryptoAmount
  recipient: '0xabc'      // optional; falls back to the bound account
})

// Sell (off-ramp)
const { sellUrl } = await transak.sell({
  cryptoAsset: 'ETH',
  fiatCurrency: 'USD',
  cryptoAmount: 100_000_000_000_000_000n, // 0.1 ETH in wei
  refundAddress: '0xabc'                  // optional
})

// Quote (no widget opened)
const quote = await transak.quoteBuy({ cryptoAsset: 'ETH', fiatCurrency: 'USD', fiatAmount: 10_000n,
// → { cryptoAmount, fiatAmount, fee (bigint base units), rate (string), metadata }
```

Provider-specific extras — `paymentMethod`, `network`, `referrerDomain`, and other [Transak widget parameters](#) — go under `config`:

```
await transak.buy({
  cryptoAsset: 'USDT', fiatCurrency: 'USD', fiatAmount: 10_000n,
  config: { network: 'tron', paymentMethod: 'credit_debit_card', referrerDomain: 'yourdomain' }
})
```

Notes

- **referrerDomain (required for buy/sell)** — Transak's Create Widget URL API requires `config.referrerDomain` (your web domain or app package name); the widget URL fails without it. It may need allow-listing in the Partner dashboard. `quoteBuy/quoteSell` don't need it.
- **Conventions** — `cryptoAsset/fiatCurrency` are upper-case (`ETH`, `USD`), `network/paymentMethod` lower-case (`ethereum`, `credit_debit_card`). They're matched exactly, with no normalisation (a wrong case throws `Cannot find info for cryptoAsset and fiatCurrency`). Fetch the exact values with the `supported-currencies` methods.
- **Multi-network assets** — a symbol like `USDT` exists on several chains; `config.network` picks one (first match if omitted).
- **Wallet address** — resolved from `recipient/refundAddress`, else the bound account's address, else the Transak widget prompts the user. `quote*`, `getTransactionDetail`, and `getSupported*` never use the account.

Manage Transactions

Get transaction details

Look up an order by its Transak ID:

```
const detail = await transak.getTransactionDetail('order-id')
// {
//   status: 'completed',           // 'in_progress' | 'failed' | 'completed'
//   cryptoAsset: 'ETH',
//   fiatCurrency: 'USD',
//   metadata: { id: 'order-id', status: 'COMPLETED', /* ...full Transak order */ }
// }
```

`status` normalises Transak's order status to a standard value; the raw Transak order is under `metadata`.

Transak's Get Order API requires a partner `access-token` (minted from your API secret), which must not be exposed client-side. So `getTransactionDetail` delegates the authenticated fetch to a `getOrder` callback that runs on your backend — the same pattern as `widgetUrl`. It's **required**; `getTransactionDetail` throws without it. See [Fetching an order \(backend\)](#).

Supported currencies and countries

Fetch the exact values your account supports at runtime (cached per `cacheTime`). Use `code/networkCode` verbatim as `cryptoAsset/network`, and `code` as `fiatCurrency`; `paymentMethod` ids are in `metadata.paymentOptions[].id`.

```
await transak.getSupportedCryptoAssets()
// [{ code: 'ETH', networkCode: 'ethereum', decimals: 18, name, metadata }, ...]

await transak.getSupportedFiatCurrencies()
// [{ code: 'USD', decimals: 2, name, metadata }, ...]

await transak.getSupportedCountries()
// [{ code: 'US', isBuyAllowed: true, isSellAllowed: true, name, metadata }, ...]
```

Configuration

`new TransakProtocol(account, config)` — the `config` object:

Option	Type	Default	Description
<code>apiKey</code>	string	—	Your Transak partner API key (Partner dashboard).
<code>widgetUrl</code>	function	—	Callback that receives the assembled <code>widgetParams</code> object and returns a widget URL. Required for buy/sell (they throw without it); the other methods don't need it.

<code>getOrder</code>	function	—	Callback <code>(txId) => Promise<TransakOrder></code> that fetches a Transak order on your backend. Required for <code>getTransactionDetail</code> (it throws without it); the other methods don't need it.
<code>environment</code>	<code>'PRODUCTION'</code> <code>'STAGING'</code>	<code>'PRODUCTION'</code>	Selects the Transak API host. Use <code>'STAGING'</code> for testing with non-real funds.
<code>cacheTime</code>	number	<code>600000</code>	Milliseconds to cache the supported crypto/fiat lists.

API Reference

Method	Description	Returns
<code>buy(options)</code>	Generates a widget URL to purchase crypto	<code>Promise<{ buyUrl }></code>
<code>sell(options)</code>	Generates a widget URL to sell crypto	<code>Promise<{ sellUrl }></code>
<code>quoteBuy(options)</code>	Gets a quote for a crypto asset purchase	<code>Promise<TransakBuyQuote></code>
<code>quoteSell(options)</code>	Gets a quote for a crypto asset sale	<code>Promise<TransakSellQuote></code>
<code>getTransactionDetail(txId)</code>	Retrieves the details of an order	<code>Promise<TransakTransactionDetail></code>
<code>getSupportedCryptoAssets()</code>	Lists supported crypto assets	<code>Promise<TransakSupportedCryptoAsset[]></code>
<code>getSupportedFiatCurrencies()</code>	Lists supported fiat currencies	<code>Promise<TransakSupportedFiatCurrency[]></code>
<code>getSupportedCountries()</code>	Lists supported countries	<code>Promise<TransakSupportedCountry[]></code>

`buy(options)` / `sell(options)`

- `cryptoAsset` (string): The crypto asset code (e.g. `'ETH'`).
- `fiatCurrency` (string): The fiat currency code (e.g. `'USD'`).
- `fiatAmount` (number | bigint, optional): The fiat amount, in base units (e.g. `10_000n` = 100 USD in cents).

- `cryptoAmount` (number | bigint, optional): The crypto amount, in base units (e.g. `1_000_000_000_000_000_000n` = 1 ETH in wei).
- `recipient` (string, optional, `buy` only): The destination wallet address. Falls back to the account address.
- `refundAddress` (string, optional, `sell` only): The refund wallet address. Falls back to the account address.
- `config` (object, optional): Additional Transak widget parameters, including `network`.

Returns `Promise<{ buyUrl }>` / `Promise<{ sellUrl }>` — the URL your `widgetUrl` callback produced, e.g. `https://global.transak.com/?apiKey=...&sessionId=...`.

`quoteBuy(options)` / `quoteSell(options)`

- `cryptoAsset` (string): The crypto asset code.
- `fiatCurrency` (string): The fiat currency code.
- `fiatAmount` (number | bigint): The fiat amount in base units. `quoteBuy` accepts this or `cryptoAmount`.
- `cryptoAmount` (number | bigint): The crypto amount in base units. Required for `quoteSell`; `quoteBuy` accepts this or `fiatAmount`.
- `config` (object, optional): Provider-specific extras — `paymentMethod` and `network` (resolved from the supported list when omitted).

Returns `Promise<TransakBuyQuote>` / `Promise<TransakSellQuote>`:

```
{
  cryptoAmount: 500000000000000000n, // base units (wei)
  fiatAmount: 100000n,                // base units (cents)
  fee: 500n,                          // base units (cents)
  rate: '2000',                       // decimal string, standard units
  metadata: { quoteId: 'q1', conversionPrice: 2000, /* ...full Transak quote */ }
}
```

`getTransactionDetail(txId)`

- `txId` (string): The Transak order ID.

Requires the `getOrder` callback (see Configuration and Fetching an order (backend)); throws without it.

Returns `Promise<TransakTransactionDetail>` — `{ status, cryptoAsset, fiatCurrency, metadata }`, where `status` is `'in_progress'` | `'failed'` | `'completed'`.

`getSupportedCryptoAssets()` / `getSupportedFiatCurrencies()` / `getSupportedCountries()`

Return arrays of, respectively:

- `{ code, networkCode, decimals, name, metadata }`
- `{ code, decimals, name, metadata }`
- `{ code, isBuyAllowed, isSellAllowed, name, metadata }`

The raw Transak object is always available under `metadata`.

Creating the widget URL (backend)

`buy` and `sell` build a `widgetParams` object and hand it to your `widgetUrl` callback. That callback runs on your backend, where your API secret is safe, and turns the params into a widget URL using Transak's [API-based flow](#) (passing params directly in the URL is deprecated). It's two calls:

```
// On your backend - never ship the API secret to the client.
// `userIp` is the end user's IP from the incoming request (e.g. req.ip, or your
// CDN's cf-connecting-ip) - Transak requires it as the `x-user-ip` header.
async function createWidgetUrl (widgetParams, userIp) {
  // 1. Get a partner access token (cache it until it expires).
  const tokenRes = await fetch('https://api.transak.com/partners/api/v2/refresh-token', {
    method: 'POST',
    headers: { 'api-secret': process.env.TRANSAK_API_SECRET, 'content-type': 'application/json' },
    body: JSON.stringify({ apiKey: process.env.TRANSAK_API_KEY })
  })
  const { data: { accessToken } } = await tokenRes.json()

  // 2. Create the widget session and return its URL.
  const sessionRes = await fetch('https://api-gateway.transak.com/api/v2/auth/session', {
    method: 'POST',
    headers: { 'access-token': accessToken, 'content-type': 'application/json', 'x-user-ip': userIp },
    body: JSON.stringify({ widgetParams })
  })
  const { data: { widgetUrl } } = await sessionRes.json()
  return widgetUrl // valid for 5 minutes, single use
}
```

Notes:

- Keep the API secret on the backend, never in client code.
 - **x-user-ip is mandatory** on Transak's authenticated APIs — send the end user's originating IP (a valid IPv4/IPv6), taken from the incoming request. See [Transak's security changes](#).
 - The returned widget URL is valid for **5 minutes** and each session can be used **once** — create a fresh one per flow.
 - For staging, use <https://api-stg.transak.com> and <https://api-gateway-stg.transak.com>.
 - For the full list of widget parameters, see the [Transak query parameters docs](#).
-

Fetching an order (backend)

`getTransactionDetail` hands an order id to your `getOrder` callback, which fetches the order on your backend. Transak's [Get Order API](#) needs a partner `access-token` (minted from your API secret, same as the widget URL flow) alongside `x-api-key`:

```
// On your backend – never ship the API secret to the client.
// `userIp` is the end user's IP (Transak requires it as the `x-user-ip` header).
async function getOrder (txId, userIp) {
  // 1. Get a partner access token (reuse the same one as the widget URL flow; cache until it
  const tokenRes = await fetch('https://api.transak.com/partners/api/v2/refresh-token', {
    method: 'POST',
    headers: { 'api-secret': process.env.TRANSAK_API_SECRET, 'content-type': 'application/json' },
    body: JSON.stringify({ apiKey: process.env.TRANSAK_API_KEY })
  })
  const { data: { accessToken } } = await tokenRes.json()

  // 2. Fetch the order and return it (the module maps its status).
  const orderRes = await fetch(`https://api.transak.com/partners/api/v2/order/${txId}`, {
    headers: { 'x-api-key': process.env.TRANSAK_API_KEY, 'access-token': accessToken, 'x-user-ip': userIp }
  })
  const { data } = await orderRes.json() // Get Order responses are wrapped in { data }
  return data
}
```

Expose this behind an endpoint that responds with `{ order }`; the browser-side `getOrder` callback calls that endpoint — the same pattern as `widgetUrl`.

Development

```
npm install          # install dependencies
npm run build:types  # build TypeScript definitions
npm run lint         # lint code
npm run lint:fix     # fix linting issues
npm test            # run tests (mock-only)
npm run test:coverage # run tests with coverage
```

Live smoke test

`npm test` is mock-only. To exercise all 8 methods against a real Transak environment, use the smoke test — copy the env template and fill it in (`.env` is gitignored):

```
cp .env.example .env # set TRANSAK_API_KEY, TRANSAK_API_SECRET,
                     # TRANSAK_REFERRER_DOMAIN (allow-listed), TRANSAK_USER_IP (a valid IP)
npm run smoke        # runs scripts/smoke-test.mjs against STAGING
```

It fetches the supported lists, requests buy/sell quotes, and generates buy/sell widget URLs (implementing the `widgetUrl/getOrder` backends inline, with the required `referrerDomain` and `x-user-ip`) — printing each method's return. Set `TRANSAK_ORDER_ID` in `.env` to also exercise `getTransactionDetail`.

License

Apache License 2.0 — see the LICENSE file for details.

Contributing

Contributions are welcome! Please feel free to submit a Pull Request.

Support

For support, please open an issue on the GitHub repository.

--	--	--	--